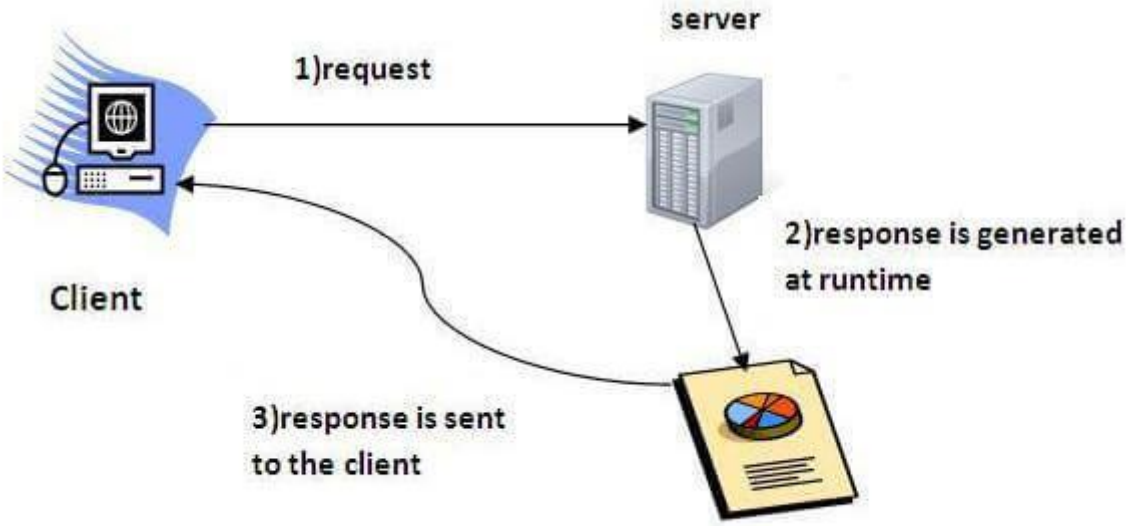
Unit II-Servlet

What is a Servlet?

- Servlet is a technology which is used to create a web application.
- Servlet is an API that provides many interfaces and classes including documentation.
- Servlet is an interface that must be implemented for creating any Servlet.
- Servlet is a class that extends the capabilities of the servers and responds to the incoming requests. It can respond to any requests.
- Servlet is a web component that is deployed on the server to create a dynamic web page.
- **Servlet** technology is used to create a web application (resides at server side and generates a dynamic web page).



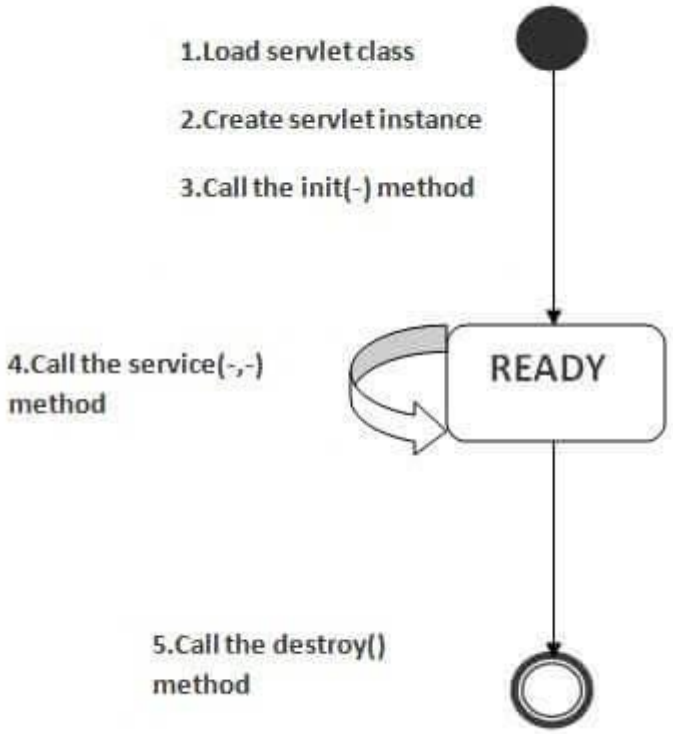
What is a web application?

A web application is an application accessible from the web. A web application is composed of web components like Servlet, JSP, Filter, etc. and other elements such as HTML, CSS, and JavaScript. The web components typically execute in Web Server and respond to the HTTP request.

Life Cycle of a Servlet

The web container maintains the life cycle of a servlet instance. Let's see the life cycle of the servlet:

1. Servlet class is loaded.
2. Servlet instance is created.
3. init method is invoked.
4. service method is invoked.
5. destroy method is invoked.



As displayed in the above diagram, there are three states of a servlet: new, ready and end. The servlet is in new state if servlet instance is created. After invoking the init() method, Servlet comes in the ready state. In the ready state, servlet performs all the tasks. When the web container invokes the destroy() method, it shifts to the end state.

1) Servlet class is loaded

The classloader is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.

---

2) Servlet instance is created

The web container creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.

---

3) init method is invoked

The web container calls the init method only once after creating the servlet instance. The init method is us initialize the servlet. It is the life cycle method of the javax.servlet.Servlet interface. Syntax of the init meth given below:

```
1. public void init(ServletConfig config) throws ServletException
```

---

4) service method is invoked

The web container calls the service method each time when request for the servlet is received. If servlet is not initialized, it follows the first three steps as described above then calls the service method. If servlet is initialized, it calls the service method. Notice that servlet is initialized only once. The syntax of the service method of the Servlet interface is given below:

```
1. public void service(ServletRequest request, ServletResponse response)
     throws ServletException, IOException
```

---

5) destroy method is invoked

The web container calls the destroy method before removing the servlet instance from the service. It gives the servlet an opportunity to clean up any resource for example memory, thread etc. The syntax of the destroy method of the Servlet interface is given below:

```
1. public void destroy()
```

What is Tomcat?

It is an open-source Java servlet container that implements many Java Enterprise Specs such as the Websites API, Java-Server Pages and last but not least, the Java Servlet. The complete name of Tomcat is "Apache Tomcat" it was developed in an open, participatory environment and released in 1998 for the very first time. It began as the reference implementation for the very first Java-Server Pages and the Java Servlet API. However, it no longer works as the reference implementation for both of these technologies, but it is considered as the first choice among the users even after that. It is still one of the most widely used java-sever due to several capabilities such as good extensibility, proven core engine, and well-test and durable. Here we used the term "servlet" many times, so what is java servlet; it is a kind of software that enables the webserver to handle the dynamic(java-based) content using the Http protocols.

Apache tomcat

If you are a little familiar with the websites or have some basic knowledge about the websites, you must have heard about the HTTP protocol or may also know what actually are they. If you want to provide any web-services such as you want to provide a simple static content possibly by using HTML (or Hypertext Markup Language), or maybe you just want to send data from a server to point you, so you necessarily need a server and that server is HTTP(HyperText transfer protocol). So, as we all know that if anyone wants to make a simple, static website, he definitely requires an HTTP server, but if he wants to make website dynamic, he has to use servlet. We use the HTTP server if we want to send simple data. If we want to send dynamic data or to make our website dynamic, we need to use the servlet. Hence, we need an HTTP server and what else we need is a container where we will run or servlet, so when we combine the HTTP server and the servlet (or we can say servlet container), they both combine to become a single server know as tomcat server.

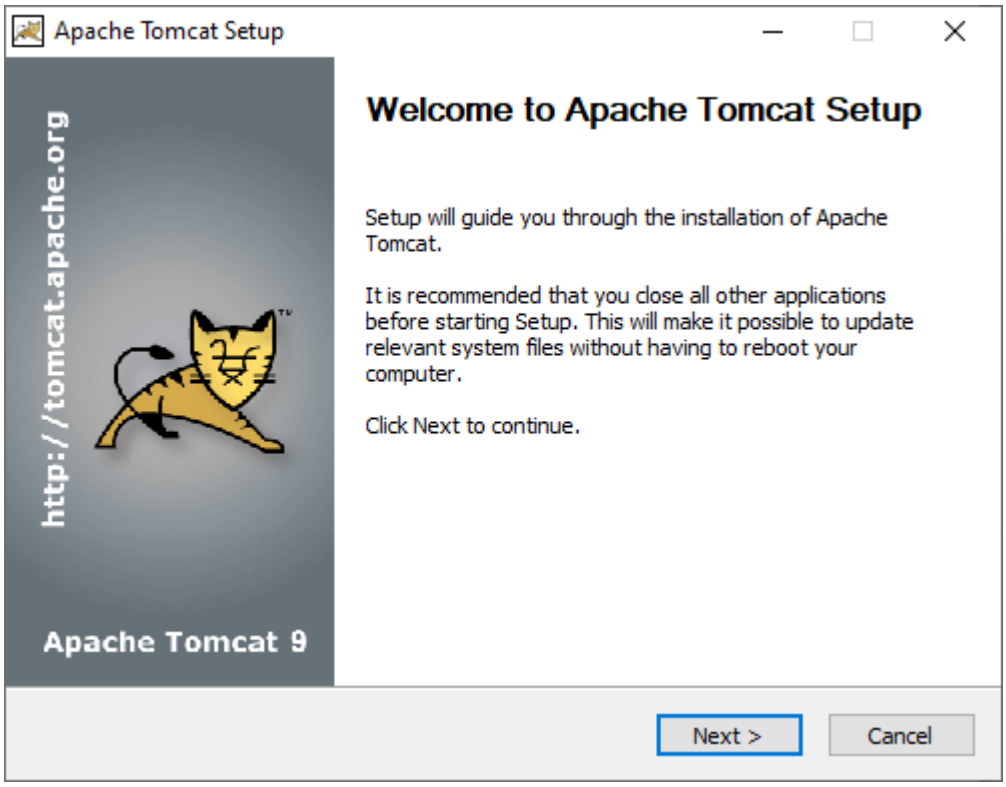
Steps to download and install the Tomcat on Windows 10 operating system

There are many versions of the Tomcat available with different features on its website. You can choose the version according to your requirements, but we suggest you download the latest stable version.

Step 1. Open the Google Chrome or any of your web browser and type "download Tomcat for windows" in the search box. You can also go directly on Tomcat's website by clicking on this https://tomcat.apache.org/download-90.cgi#9.0.38

Step 2. Go to Download and click on the downloaded file and wait for little until the installation process starts.

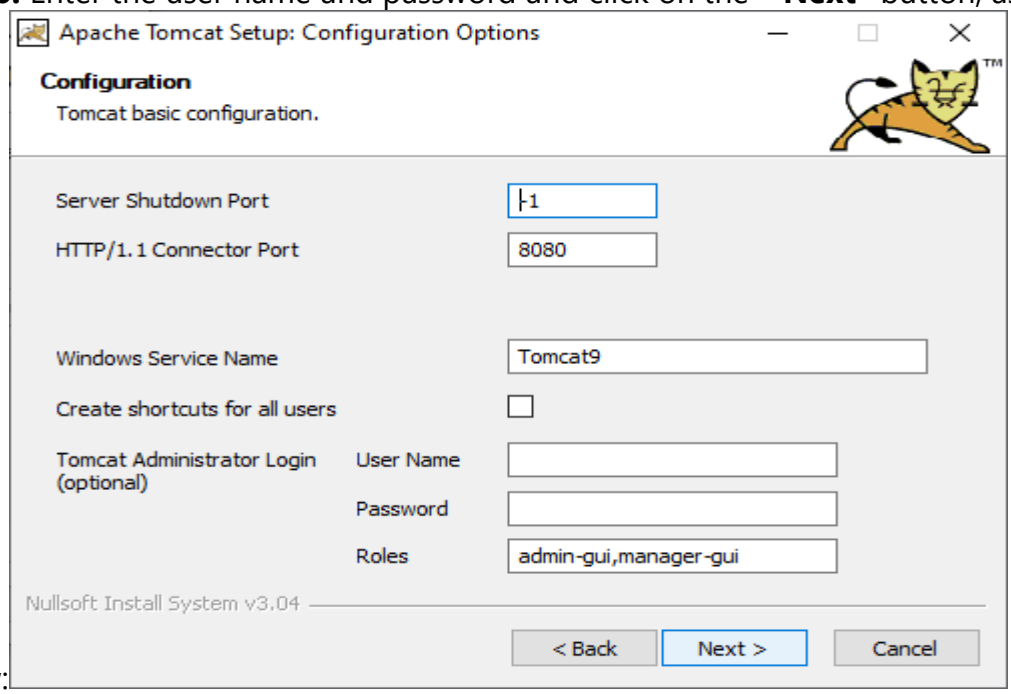
Step 3. Once the installation process gets started, click on the **"Next"** button, as shown below:



Step 4. Click on the button labeled as **"I Agree."**

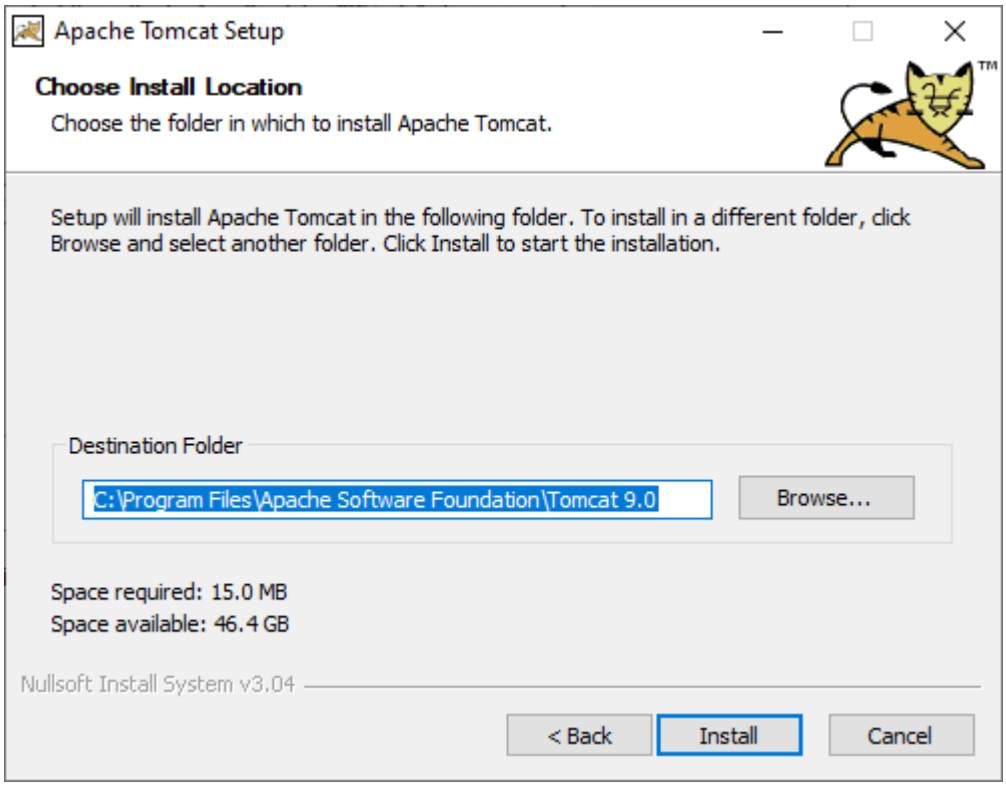
Step 5. Click on the **"Next"** button.

Step 6. Enter the user name and password and click on the **" Next"** button, as shown below:



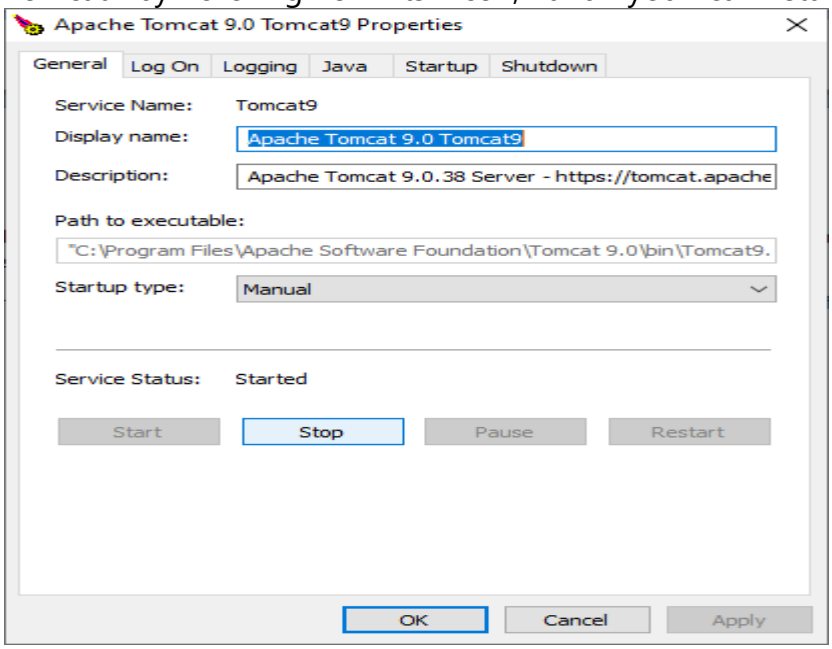
Step 7. Then click on the **"Next"** button again

Step 8. Now click on the **"Install"** button.



Wait for some time until the Tomcat gets installed.

Step 9. Now click on the **"Finish"** button, here the installation of Tomcat is completed. It may ask you to restart your system, so restart your system.Now you can start the Tomcat by clicking on its icon, and you can start and stop the server.



Servlet API

The javax.servlet and javax.servlet.http packages represent interfaces and classes for servlet api.

The **javax.servlet** package contains many interfaces and classes that are used by the servlet or web container. These are not specific to any protocol.

The **javax.servlet.http** package contains interfaces and classes that are responsible for http requests only.

Let's see what are the interfaces of javax.servlet package.

Interfaces in javax.servlet package

There are many interfaces in javax.servlet package. They are as follows:

1. Servlet
2. ServletRequest
3. ServletResponse
4. RequestDispatcher
5. ServletConfig
6. ServletContext
7. SingleThreadModel
8. Filter
9. FilterConfig
10. FilterChain
11. ServletRequestListener
12. ServletRequestAttributeListener
13. ServletContextListener
14. ServletContextAttributeListener

Classes in javax.servlet package

There are many classes in javax.servlet package. They are as follows:

1. GenericServlet
2. ServletInputStream

3. ServletOutputStream
4. ServletRequestWrapper
5. ServletResponseWrapper
6. ServletRequestEvent
7. ServletContextEvent
8. ServletRequestAttributeEvent
9. ServletContextAttributeEvent
10. ServletException
11. UnavailableException

Interfaces in javax.servlet.http package

There are many interfaces in javax.servlet.http package. They are as follows:

1. HttpServletRequest
2. HttpServletResponse
3. HttpSession
4. HttpSessionListener
5. HttpSessionAttributeListener
6. HttpSessionBindingListener
7. HttpSessionActivationListener
8. HttpSessionContext (deprecated now)

Classes in javax.servlet.http package

There are many classes in javax.servlet.http package. They are as follows:

1. HttpServlet
2. Cookie
3. HttpServletRequestWrapper
4. HttpServletResponseWrapper
5. HttpSessionEvent
6. HttpSessionBindingEvent
7. HttpUtils (deprecated now)

Handling http request and response with HTTPGET and HTTPPOST.

HTTPGET

The GET method is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data.

The HttpClient API provides a class named **HttpGet** which represents the get request method.

Follow the steps given below to send a get request using HttpClient library

Step 1 - Create a HttpClient object

The **createDefault()** method of the **HttpClients** class returns a **CloseableHttpClient** object, which is the base implementation of the **HttpClient** interface.

Using this method, create an HttpClient object as shown below −

```
CloseableHttpClient httpclient = HttpClients.createDefault();
```

Step 2 - Create an HttpGet Object

The **HttpGet** class represents the HTTPGET request which retrieves the information of the given server using a URI.

Create a HTTP GET request by instantiating this class. The constructor of this class accepts a String value representing the URI.

```
HttpGet httpget = new HttpGet("https://www.tutorialspoint.com/");
```

Step 3 - Execute the Get Request

The **execute()** method of the **CloseableHttpClient** class accepts a HttpUriRequest (interface) object (i.e. HttpGet, HttpPost, HttpPut, HttpHead etc.) and returns a response object.

Execute the request using this method as shown below −

```
HttpResponse httpresponse = httpclient.execute(httpget);
```

HTTPPOST

A POST request is used to send data to the server; for example, customer information, file upload, etc., using HTML forms.

The HttpClient API provides a class named **HttpPost** which represents the POST request.

Follow the steps given below to send a HTTP POST request using HttpClient library.

Step 1 - Create an HttpClient Object

The **createDefault()** method of the **HttpClients** class returns an object of the class **CloseableHttpClient**, which is the base implementation of the **HttpClient** interface.

Using this method, create an HttpClient object.

```
CloseableHttpClient httpClient = HttpClients.createDefault();
```

Step 2 - Create HttpPost Object

The **HttpPost** class represents the **HTTP POST** request. This sends required data and retrieves the information of the given server using a URI.

Create this request by instantiating the **HttpPost** class and pass a string value representing the URI, as a parameter to its constructor.

```
HttpGet httpGet = new HttpGet("https://www.tutorialspoint.com/");
```

Step 3 - Execute the Get Request

The **execute()** method of the CloseableHttpClient object accepts a HttpUriRequest (interface) object (i.e. HttpGet, HttpPost, HttpPut, HttpHead etc.) and returns a response object.

```
HttpResponse httpResponse = httpclient.execute(httpget);
```

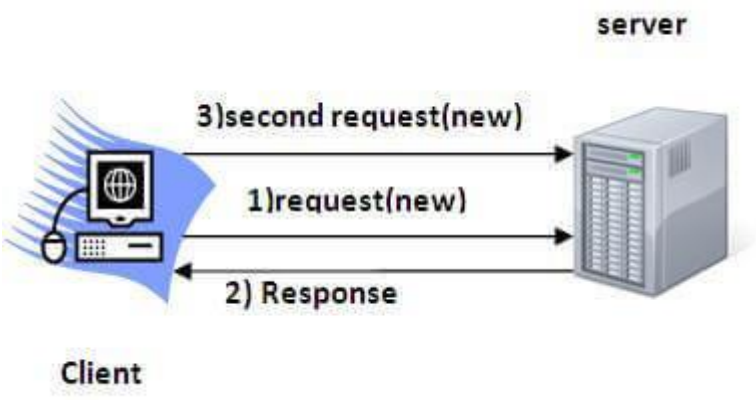
Session Tracking in Servlets

Session simply means a particular interval of time.

Session Tracking is a way to maintain state (data) of an user. It is also known as **session management** in servlet.

Http protocol is a stateless so we need to maintain state using session tracking techniques. Each time user requests to the server, server treats the request as the new request. So we need to maintain the state of an user to recognize to particular user.

HTTP is stateless that means each request is considered as the new request. It is shown in the figure given below:



Why use Session Tracking?

To recognize the user It is used to recognize the particular user.

Session Tracking Techniques

There are four techniques used in Session tracking:

1. **Cookies**
2. **Hidden Form Field**
3. **URL Rewriting**
4. **HttpSession**

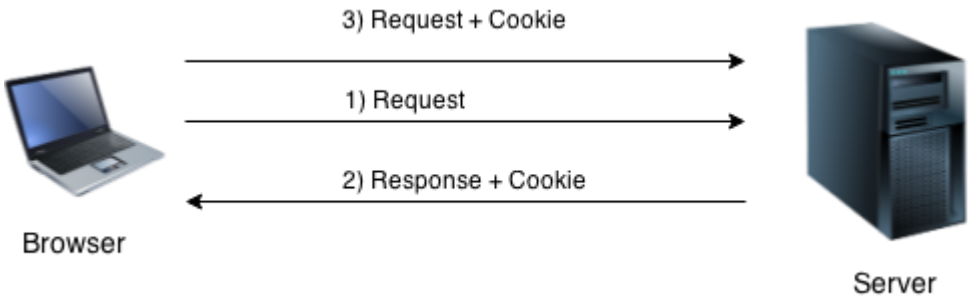
Cookies in Servlet

A **cookie** is a small piece of information that is persisted between the multiple client requests.

A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

How Cookie works

By default, each request is considered as a new request. In cookies technique, we add cookie with response from the servlet. So cookie is stored in the cache of the browser. After that if request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.



Types of Cookie

There are 2 types of cookies in servlets.

1. Non-persistent cookie
2. Persistent cookie

Non-persistent cookie

It is **valid for single session** only. It is removed each time when user closes the browser.

Persistent cookie

It is **valid for multiple session** . It is not removed each time when user closes the browser. It is removed only if user logout or signout.

Advantage of Cookies

1. Simplest technique of maintaining the state.
2. Cookies are maintained at client side.

Disadvantage of Cookies

1. It will not work if cookie is disabled from the browser.
2. Only textual information can be set in Cookie object.

Java Server Pages(JSP)

JSP technology is used to create web application just like Servlet technology. It can be thought of as an extension to Servlet because it provides more functionality than servlet such as expression language, JSTL, etc.

A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than Servlet because we can separate designing and development. It provides some additional features such as Expression Language, Custom Tags, etc.

Advantages of JSP over Servlet

There are many advantages of JSP over the Servlet. They are as follows:

1) Extension to Servlet

JSP technology is the extension to Servlet technology. We can use all the features of the Servlet in JSP. In addition to, we can use implicit objects, predefined tags, expression language and Custom tags in JSP, that makes JSP development easy.

2) Easy to maintain

JSP can be easily managed because we can easily separate our business logic with presentation logic. In Servlet technology, we mix our business logic with the presentation logic.

3) Fast Development: No need to recompile and redeploy

If JSP page is modified, we don't need to recompile and redeploy the project. The Servlet code needs to be updated and recompiled if we have to change the look and feel of the application.

4) Less code than Servlet

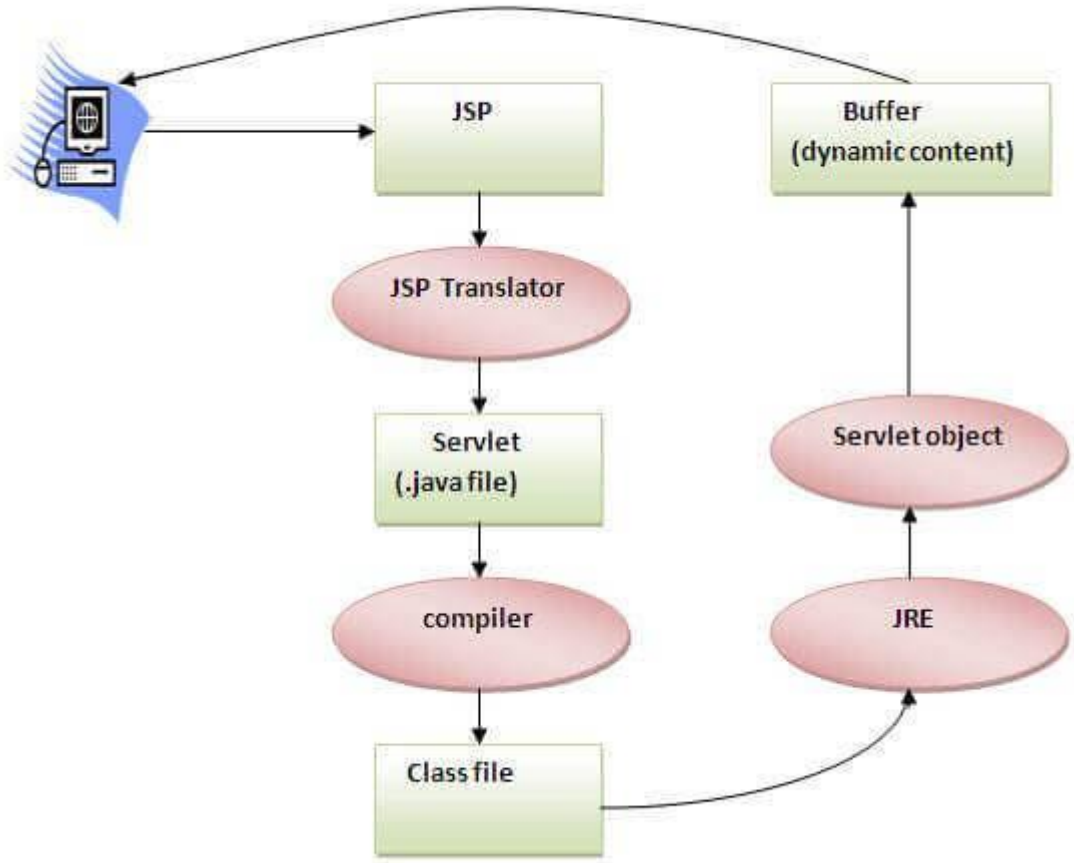
In JSP, we can use many tags such as action tags, JSTL, custom tags, etc. that reduces the code. Moreover, we can use EL, implicit objects, etc.

The Lifecycle of a JSP Page

The JSP pages follow these phases:

- Translation of JSP Page
- Compilation of JSP Page
- Classloading (the classloader loads class file)
- Instantiation (Object of the Generated Servlet is created).
- Initialization (the container invokes jspInit() method).
- Request processing (the container invokes _jspService() method).
- Destroy (the container invokes jspDestroy() method).

Note: jspInit(), _jspService() and jspDestroy() are the life cycle methods of JSP.



As depicted in the above diagram, JSP page is translated into Servlet by the help of JSP translator. The JSP translator is a part of the web server which is responsible for translating the JSP page into Servlet. After that, Servlet page is compiled by the compiler and gets converted into the class file. Moreover, all the processes that happen in Servlet are performed on JSP later like initialization, committing response to the browser and destroy.

JSP tags

There are main 3 types of tags

- scriptlet tag
- expression tag
- declaration tag

JSP scriptlet tag

A scriptlet tag is used to execute java source code in JSP. Syntax is as follows:

```
1. <%  java source code %>
```

Example of JSP scriptlet tag

In this example, we are displaying a welcome message.

```
1. <html>
2. <body>
3. <% out.print("welcome to jsp"); %>
4. </body>
5. </html>
```

Example of JSP scriptlet tag that prints the user name

In this example, we have created two files index.html and welcome.jsp. The index.html file gets the username from the user and the welcome.jsp file prints the username with the welcome message.

File: index.html

```
<html>
<body>
<form action="welcome.jsp">
```

```
<input type="text" name="uname">
    <input type="submit" value="go"><br/>
</form>
</body>
    </html>
```

JSP expression tag

The code placed within **JSP expression tag** is *written to the output stream of the response*. So you need not write out.print() to write data. It is mainly used to print the values of variable or method.

Syntax of JSP expression tag

1. <%= statement %>

Example of JSP expression tag

In this example of jsp expression tag, we are simply displaying a welcome message.

```
<html>
<body>
<%= "welcome to jsp" %>
</body>
</html>
```

Example of JSP expression tag that prints current time

To display the current time, we have used the getTime() method of Calendar class. The getTime() is an instance method of Calendar class, so we have called it after getting the instance of Calendar class by the getInstance() method.

```
<html>
<body>
Current Time: <%= java.util.Calendar.getInstance().getTime() %>
</body>
</html>
```

JSP Declaration Tag

The **JSP declaration tag** is used *to declare fields and methods*.

The code written inside the jsp declaration tag is placed outside the service() method of auto generated servlet.

So it doesn't get memory at each request.

Syntax of JSP declaration tag

The syntax of the declaration tag is as follows:

1. <%! field or method declaration %>

Example of JSP declaration tag that declares field

In this example of JSP declaration tag, we are declaring the field and printing the value of the declared field using the jsp expression tag.

index.jsp

```
<html>
<body>
<%! int data=50; %>
<%= "Value of the variable is:"+data %>
</body>
</html>
```

Application using JSP and Servlet.

Index.jsp

```
<%--
   Document   : newjsp
   Created on : 29 Jul, 2022, 10:28:24 AM
   Author     : hp
--%>


<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
```

```
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">


   <!-- Compiled and minified CSS -->
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/css/materialize.min.css">


  <!-- Compiled and minified JavaScript -->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/js/materialize.min.js"></script>
  </head>
  <body style="background: url(img/pexels-suneo-103573.jpg);background-size: cover;background-
attachment: fixed">
    <div class= "container" >
      <div class="row">
      <div class="col m6offset-m3">
      <div class="card">
        <div class="card_content">
      <h3 style = "margin-top: 35px;">Register here !!</h3>
          <div class="row" >


      <div class="form center_align">
        <form action="Register" method="post" id="myform">
          <input type="text" name="user_name" placeholder="Enter user name"/>
          <input type="password" name="user_pass" placeholder="Enter user password"/>
          <input type="email" name="user_email" placeholder="Enter user email id"/>
           <div class="file-field input-field">
                <div class="btn">
                <span>File</span>
                <input name="image" type="file">
                 </div>
```

```
                <div class="file-path-wrapper">
                <input class="file-path validate" type="text">
                </div>
                    </div>
            <button type="submit" class="btn red"> SUBMIT </button>
          </form>
        </div>
        <div class="loader center_align" style="margin-top: 10px ">
            <div class="preloader-wrapper big active">
  <div class="spinner-layer spinner-blue-only">
    <div class="circle-clipper left">
      <div class="circle"></div>
    </div><div class="gap-patch">
      <div class="circle"></div>
    </div><div class="circle-clipper right">
      <div class="circle"></div>
    </div>
  </div>
</div>

<div class="preloader-wrapper active">
  <div class="spinner-layer spinner-red-only">
    <div class="circle-clipper left">
      <div class="circle"></div>
    </div><div class="gap-patch">
      <div class="circle"></div>
    </div><div class="circle-clipper right">
      <div class="circle"></div>
    </div>
  </div>
</div>
```

```
<div class="preloader-wrapper small active">
  <div class="spinner-layer spinner-green-only">
    <div class="circle-clipper left">
      <div class="circle"></div>
    </div><div class="gap-patch">
      <div class="circle"></div>
    </div><div class="circle-clipper right">
      <div class="circle"></div>
    </div>
  </div>
</div>
<h5> please wait.....</h5>
</div>


   <script
src="https://code.jquery.com/jquery-migrate-3.4.0.js"
integrity="sha256-0Nkb10Hnhm4EJZ0QDpvInc3bRp77wQIbIQmWYH3Y7Vw="
crossorigin="anonymous"></script>

<script>
    $(document).ready(function(){
        console.log("page is ready")

        $("#myform").on('submit',function (Event){
```

```
  Event.preventDefault();

 // var f = $(this).serialize();
   let f=new FormData(this);
  console.log(f);

$(".loader").show();
$(".form").hide();

$.ajax({
   url: "Register",
   data: f,
   type: 'POST',
   success: function (data, textStatus, jqXHR) {
       console.log(data);
       console.log("succes.........")
       $(".loader").hide();
       $(".form").show();
       if(data.trim()===='done')
       {
          $('#msg').html("Successfully Registered")
          $('#msg').addclass('green_text')
       }else
       {
          $('msg').html("somthing went wrong on server...")
           $('#msg').addclass('red_text')
       }

      },
   error: function (jqXHR, textStatus, errorThrown) {
      console.log(data);
```

```
                console.log("error.....")
                $(".loader").hide();
                $(".form").show();
            }
        }
        }
      processData:false,
      contentType:false
    } )
 </script>
            ,
  </body>
</html>
```

Register.servlet

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package com.user;

import static com.sun.org.apache.xerces.internal.util.FeatureState.is;
import java.io.File;
import java.io.IOException;
import java.io.InputStream;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
```

```
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.servlet.ServletException;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.Part;

/**
 *
 * @author hp
 */
@MultipartConfig
public class Register extends HttpServlet {

    /**
     * Processes requests for both HTTP <code>GET</code> and <code>POST</code>
     * methods.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException, ClassNotFoundException, SQLException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
```

```
/* TODO output your page here. You may use following sample code. */
// out.println("<!DOCTYPE html>");
//out.println("<html>");
//out.println("<head>");
//out.println("<title>Servlet Register</title>");
//out.println("</head>");
//out.println("<body>");

//getting all the incoming details
String name =request.getParameter("user_name");
String password =request.getParameter("user_pass");
String email =request.getParameter("user_email");
//   String filename =Part.getSubmittedFileName();
// out.println(filename);
// out.println(name);
//out.println(password);
//out.println(email);

//connection
try
{
Thread.sleep(3000);

Class.forName("com.mysql.jdbc.Driver");
Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/student","root","");
String q="insert into imp(name,password,email,imageName) values(?,?,?,?)";

PreparedStatement pst=con.prepareStatement(q);
pst.setString(1,name);
pst.setString(2,password);
```

```
                pst.setString(3,email);
                 pst.setString(4,"fileName");
              //   pst.setString(4,fileName);
                pst.executeUpdate();

              // InputStream is = part.getInputStream();
              // byte[]data=new byte[is.avalable];

              // is.read(data);
              // String path=request.getRealPath("/")+"image"+File.separator+filename;
              // out.println(path);
              // FileoutputStream fos=new FileoutputStream(path);
               //fos.write(data);
              // fos.close();
                out.println("<h1> Inserted successfully </h1>");
            }catch(Exception e){
            e.printStackTrace();
            out.println("<h1>error.......</h1>");
            }

          //out.println("</body>");
          //out.println("</html>");
        }
    }

    // <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left
to edit the code.">
    /**
     * Handles the HTTP <code>GET</code> method.
     *
     * @param request servlet request
```

```
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs
 */
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    try {
        processRequest(request, response);
    } catch (ClassNotFoundException ex) {
        Logger.getLogger(Register.class.getName()).log(Level.SEVERE, null, ex);
    } catch (SQLException ex) {
        Logger.getLogger(Register.class.getName()).log(Level.SEVERE, null, ex);
    }
}

/**
 * Handles the HTTP <code>POST</code> method.
 *
 * @param request servlet request
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs
 */
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    try {
        processRequest(request, response);
    } catch (ClassNotFoundException ex) {
        Logger.getLogger(Register.class.getName()).log(Level.SEVERE, null, ex);
```

```
        } catch (SQLException ex) {
            Logger.getLogger(Register.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    /**
     * Returns a short description of the servlet.
     *
     * @return a String containing servlet description
     */
    @Override
    public String getServletInfo() {
        return "Short description";
    }// </editor-fold>

}
```